جزوه دوره مهندسی داده و مهندسی هوش مصنوعی

Data Engineering & AI Engineering

یادداشت‌های آموزشی، مفاهیم فنی و تمرین‌های دوره



جلسه پنجم : دیکشنری مهندسی داده
مدرس :محمدمهدی شکری
تاریخ :یکشنبه ۰۲ آبان ۱۴۰۵

تاریخ به روز رسانی : ۲۰۲۶۱۰۲۵

درباره این جزوه

این جزوه با هدف ثبت، سازمان‌دهی و بازنویسی آموزشی مطالب دوره رایگان «مهندسی داده و مهندسی هوش مصنوعی» تهیه شده است.

محتوای آن بر اساس توضیحات مدرس، مثال‌های مطرح‌شده در کلاس و پرسش‌وپاسخ‌های آموزشی تدوین می‌شود. تلاش شده است مطالب گفتاری کلاس به متنی منظم، قابل فهم و مناسب مطالعه مستقل تبدیل شوند، بدون اینکه مفهوم اصلی صحبت‌های مدرس تغییر کند.

هدف این جزوه صرفاً خلاصه‌کردن جلسات نیست؛ بلکه ایجاد یک منبع آموزشی است که دانشجو بتواند در طول دوره و حتی چند ماه پس از پایان آن، برای یادگیری دوباره مفاهیم، مرور نکات فنی و آماده‌شدن برای فعالیت‌های عملی به آن مراجعه کند.

رویکرد آموزشی و شیوه تدوین

در تدوین مطالب، ابتدا مفاهیم اصلی هر جلسه با زبانی ساده و دقیق توضیح داده می‌شوند. سپس مثال‌های مدرس، کاربردهای عملی و ارتباط موضوعات با مسائل واقعی مهندسی داده و هوش مصنوعی بررسی می‌شوند.

برای حفظ دقت علمی و تمایز میان محتوای اصلی کلاس و توضیحات تکمیلی، از ساختار زیر استفاده شده است:

- **مطالب اصلی جلسه :**بازنویسی منظم توضیحات مدرس با حفظ مفهوم و مثال‌های اصلی.
- **تکمیل فنی :**توضیحات دقیق‌تر، اصلاح برداشت‌های احتمالی و نکات تکمیلی بر اساس منابع فنی معتبر.
- **مثال اضافی :**مثال‌ها و سناریوهایی که برای درک بهتر مفاهیم به جزوه افزوده شده‌اند.
- **نکات کاربردی :**ملاحظات اجرایی، محدودیت‌ها و تجربه‌های مرتبط با استفاده عملی از مفاهیم.
- **جمع‌بندی و سوال‌های مرور :**مرور مهم‌ترین مطالب و ارزیابی میزان یادگیری در پایان هر جلسه.

اصطلاحات تخصصی مانندData Pipeline ، Data Warehouse، ETL، Orchestrationو سایر مفاهیم فنی با نام اصلی انگلیسی حفظ شده‌اند تا مطالعه مستندات رسمی و منابع تخصصی برای دانشجویان آسان‌تر باشد.

محدوده و توسعه جزوه

این مجموعه به‌صورت تدریجی و هم‌زمان با انتشار جلسات جدید تکمیل می‌شود و دو مسیر اصلی دوره، یعنی Data Engineering و AI Engineering، را پوشش می‌دهد.

نسخه‌های به‌روزشده جزوه‌ها، منابع تکمیلی و یادداشت‌های هر جلسه از طریق مخزن عمومی GitHub در دسترس خواهند بود.

مخزن جزوه‌ها:

https://github.com/LaneZero/data-ai-engineering-course

وب‌سایت اصلی دوره:

https://mostafaghadimi.github.io/data-course/

یادداشت درباره اعتبار محتوا

این جزوه یک مجموعه آموزشی مستقل و غیررسمی است و وابستگی رسمی به مدرس یا برگزارکنندگان دوره ندارد.

حقوق محتوای اصلی دوره متعلق به صاحبان آن است. توضیحات تکمیلی و مثال‌های افزوده‌شده نیز لزوماً بیانگر دیدگاه یا تأیید مدرس نیستند.

با وجود تلاش برای حفظ دقت فنی، توصیه می‌شود هنگام اجرای پروژه‌های واقعی، اطلاعات حساس یا تصمیم‌گیری‌های فنی مهم، مستندات رسمی ابزارها و منابع اصلی نیز بررسی شوند.

هدف از انتشار این مجموعه، **کمک به یادگیری عمیق‌تر، مرور منظم مفاهیم و اشتراک دانش با علاقه‌مندان حوزه داده و هوش مصنوعی** است.

فهرست مطالب

جلسه ۵ : دیکشنری مهندسی داده ۱

۱. معرفی جلسه و اهداف یادگیری ۲

۲. چرخه مهندسی داده و مفاهیم پایه ۲

۲.۱ چرخه عمر مهندسی داده ۲

۲.۲ کلان‌داده و شش ویژگی آن ۳

۲.۳ Metadata یا فراداده ۳

۳. دریافت، تبدیل و انتقال داده ٤

۳.۱ Data Pipeline و Data Ingestion ٤

۳.۲ Transformation و Loading ٤

۳.۳ مقایسه ETL و ELT ٤

۳.۴ Change Data Capture یا CDC ۵

۳.۵ Idempotency و Backfill ۵

۳.۶ Orchestration ٦

۴. پردازش داده و معماری‌های جریانی ٦

۴.۱ Batch Processing و Stream Processing ٦

۴.۲ معماری Kappa و Lambda ۷

۴.۳ Message Broker, Producer و Consumer ۷

۴.۴ Sink و Delivery Semantics ۸

۴.۵ Offset و Checkpoint ۸

۵. پایگاه داده و لایه‌های ذخیره‌سازی ۹

۵.۱ SQL و پایگاه داده رابطه‌ای ۹

۵.۲ NoSQL و ACID در برابر BASE ۹

۵.۳ Data Warehouse و Data Lake ۱۰

۵.۴ Data Lakehouse ۱۰

۵.۵ Row-oriented و Column-oriented Storage ۱۰
۵.۶ قالب‌های CSV, JSON, Avro, Parquet و ORC ۱۱
۵.۷ Query Engine و کلاینت پایگاه داده ۱۲
۶. طراحی مدل تحلیلی ۱۲
۶.۱ Normalization و Denormalization ۱۲
۶.۲ Data Modeling, Fact و Dimension ۱۲
۶.۳ Star Schema و Snowflake Schema ۱۳
۶.۴ Slowly Changing Dimension یا SCD ۱۳
۶.۵ Data Mart ۱٤
۷. مقیاس‌پذیری و سامانه‌های توزیع‌شده ۱٤
۷.۱ Distributed Systems و Cluster ۱٤
۷.۲ Vertical و Horizontal Scaling ۱٤
۷.۳ Partitioning, Sharding و Replication ۱۵
۷.۴ قضیه CAP و Eventual Consistency ۱۵
۸.کیفیت، حاکمیت و عملیات داده ۱٦
۸.۱ Data Quality ۱٦
۸.۲ Reconciliation و مسیر عیب‌یابی داده ۱٦
۸.۳ Governance, Lineage و Data Catalog ۱۷
۸.۴ DataOps و Version Control ۱۷
۹. انتقال تجربه مدرس و تمرین ۱۷
۱۰. نقشه ذهنی جلسه و جمع‌بندی ۱۸
۱۱. واژه‌نامه جلسه ۱۸
۱۲. سوال‌های مرور ۲۰
۱۳. پاسخ‌نامه مختصر ۲۱

جلسه ۵ : دیکشنری مهندسی داده

تاریخ :(۲۴ اکتبر ۲۰۲۶) ۰۲ آبان ۱۴۰۵

دوره :مهندسی داده و مهندسی هوش مصنوعی

مدرس جلسه :نیوشا بقایی

راهنمای مطالعه :متن اصلی، روایت آموزشی صحبت‌های مدرس است .هرجا توضیح یا اصلاحی فراتر از صحبت کلاس لازم بوده، با عنوان **تکمیل فنی** «یا» **مثال اضافی** مشخص شده است .این جلسه آموزشی و مقدماتی است؛ قرار نیست همه ابزارهایی را که نام برده شده‌اند همین حالا یاد بگیریم.

۱. معرفی جلسه و اهداف یادگیری

مدرس در ابتدای کلاس توضیح داد که هدف این جلسه، شناخت اصطلاحات پرکاربرد مهندسی داده است و قرار نیست در یک جلسه همه مفاهیم را با عمق تخصصی بررسی کنیم. این واژه‌ها در جلسات بعدی، تمرین‌ها و محیط کار بارها تکرار می‌شوند؛ بنابراین بهتر است پیش از ورود به ابزارها، تصویر درستی از ارتباط میان آن‌ها داشته باشیم.

در پایان جلسه انتظار می‌رود بتوانیم چرخه مهندسی داده را توضیح دهیم، اصطلاحات ورود و پردازش داده را از هم تشخیص دهیم، درباره انتخاب معماری و ذخیره‌سازی سوال درست بپرسیم، ساختار اولیه انبار داده را بشناسیم و مفاهیم پایه پایداری، مقیاس‌پذیری و حاکمیت داده را در گفت‌وگوهای فنی تشخیص دهیم.

۲. چرخه مهندسی داده و مفاهیم پایه

۲.۱ چرخه عمر مهندسی داده

مهندسی داده (Data Engineering) بخشی از مسیر حرکت داده از منبع به مصرف‌کننده است. ابتدا داده در سیستم‌های عملیاتی، فایل‌ها، APIها یا رویدادهای نرم‌افزاری تولید می‌شود. مهندس داده آن را دریافت می‌کند، در صورت نیاز شکل و کیفیتش را اصلاح می‌کند، در محل مناسب ذخیره می‌کند و برای تحلیل، گزارش‌گیری یا سیستم‌های هوش مصنوعی در دسترس قرار می‌دهد.

مدرس تأکید کرد که ایجاد داده در تمام منابع و استفاده نهایی از خروجی، الزاماً جزو وظایف مستقیم تیم مهندسی داده نیست. در شرکت‌های واقعی مرز نقش‌ها می‌تواند همپوشانی داشته باشد. نرم‌افزار، امنیت، Orchestration, DataOps و Data Management نیز در طراحی و نگهداری این چرخه حضور دارند.

مرحله	سوال اصلی	نمونه
Data Generation	داده کجا ایجاد می‌شود؟	ثبت سفارش فروشگاه
Ingestion	داده چگونه دریافت می‌شود؟	خواندن API یا پایگاه داده
Transformation	داده به چه شکلی تبدیل می‌شود؟	یکسان‌سازی واحد مبلغ و قالب تاریخ
Storage / Load	داده کجا نگهداری می‌شود؟	بارگذاری در انبار داده یا دریاچه داده
Serving / Consumption	چه کسی از داده استفاده می‌کند؟	تیم تحلیل، محصول یا AI

۲.۲ کلان‌داده و شش ویژگی آن

مدرس Big Data را با شش ویژگی معروف آن معرفی کرد. در منابع مختلف گاهی تنها سه ویژگی ابتدایی گفته می‌شوند. این ویژگی‌ها عبارت‌اند از Volume (حجم)، Velocity (سرعت ورود)، Variety (تنوع)، Veracity (درستی و اعتمادپذیری)، Variability (تغییرپذیری) و Value (ارزش).

ویژگی	معنی ساده	نمونه یا پرسش
Volume	مقدار داده	آیا حجم به چندین ترابایت یا بیشتر می‌رسد؟
Velocity	سرعت تولید و ورود	آیا رویدادها پیوسته و سریع وارد می‌شوند؟
Variety	گوناگونی قالب	آیا با JSON، فایل، جدول و رویداد روبه‌رو هستیم؟
Veracity	میزان اعتماد به داده	آیا داده ناقص یا ناسازگار است؟
Variability	تغییر الگو و نرخ داده	آیا ساختار یا شدت بار در زمان تغییر می‌کند؟
Value	ارزش قابل استخراج	آیا پردازش داده به تصمیم مفید منجر می‌شود؟

در گفت‌وگوی کلاس به این نکته اشاره شد که حتی داده‌ای با حجم نه‌چندان زیاد، ممکن است به سبب سرعت یا تنوع زیاد، مسئله‌ای شبیه مسائل Big Data ایجاد کند.

> **تکمیل فنی:** داشتن یکی از شش ویژگی به‌تنهایی به معنای الزام به استفاده از Spark یا زیرساخت توزیع‌شده نیست. انتخاب راه‌حل به اندازه واقعی داده، SLA، هزینه، پیچیدگی عملیاتی و توان ابزارهای ساده‌تر هم بستگی دارد. ابتدا با اندازه‌گیری و آزمایش، گلوگاه را مشخص کنید.

۲.۳ Metadata یا فراداده

Metadata اطلاعاتی درباره داده است، نه لزوماً متن یا مقدار خود داده. برای نمونه، فایل `report.pdf` محتوای گزارش را دارد؛ اما نام، نوع، حجم، زمان ایجاد و مالک فایل از جنس Metadata هستند. در سامانه داده، نام جدول، ساختار ستون‌ها، زمان به‌روزرسانی و مالک مجموعه‌داده نیز فراداده محسوب می‌شوند.

۳. دریافت، تبدیل و انتقال داده

۳.۱ Data Pipeline و Data Ingestion

Data Pipeline (خط لوله داده) مجموعه مرحله‌های متصل به هم برای انتقال و آماده‌سازی داده است. مدرس از تشبیه لوله استفاده کرد: داده از یک سمت وارد می‌شود و در مسیر، عملیات لازم روی آن انجام می‌شود تا به مقصد برسد. هدف بسیاری از این خط لوله‌ها اجرای خودکار و ارائه خروجی به‌روز است.

Data Ingestion مرحله دریافت داده از Source است؛ برای مثال خواندن سفارش‌ها از پایگاه داده سایت یا دریافت رویدادهای ثبت سفارش. منبع ممکن است متعلق به تیم دیگری یا حتی سازمانی بیرونی باشد. دریافت داده می‌تواند دوره‌ای یا جریانی باشد.

۳.۲ Transformation و Loading

Transformation (تبدیل داده) یعنی تغییر داده ورودی به شکلی که با نیاز مقصد و قواعد تعریف‌شده سازگار شود. مثال مدرس تبدیل مبلغ از ریال به تومان و یکدست‌کردن نمایش تاریخ بود. Loading (بارگذاری) قراردادن داده در سامانه مقصد است.

> **تکمیل فنی:** یکسان‌سازی تاریخ صرفاً عوض‌کردن جداکننده `/` و `-` نیست؛ باید نوع تقویم، منطقه زمانی، نوع داده و مفهوم تاریخ رویداد مشخص باشد. تبدیل ریال به تومان نیز باید با قرارداد واحد مالی و قواعد گردکردن هماهنگ باشد.

۳.۳ مقایسه ETL و ELT

ETL مخفف Extract, Transform و Load است: ابتدا داده دریافت می‌شود، سپس تبدیل انجام می‌شود و نتیجه در مقصد بارگذاری می‌گردد. در ELT، داده ابتدا استخراج و در یک محل مناسب بارگذاری می‌شود و تبدیل‌ها پس از آن انجام می‌گیرند. انتخاب میان این دو به معماری، حساسیت داده و هدف استفاده وابسته است.

معیار	ETL	ELT
ترتیب	Extract → Transform → Load	Extract → Load → Transform
محل معمول تبدیل	پردازش پیش از مقصد نهایی	محیط مقصد یا لایه پردازشی نزدیک آن

معیار	ETL	ELT
مزیت متداول	کنترل بیشتر بر خروجی قبل از بارگذاری	نگهداری داده خام و امکان تبدیل‌های گوناگون
سوال طراحی	آیا مقصد فقط داده تبدیل‌شده می‌پذیرد؟	آیا نگهداری نسخه خام ارزشمند است؟

مدرس مثال زد که گاهی یک سازمان ساختار انبار داده را از قبل تعریف کرده و لازم است داده پیش از ورود به آن تبدیل شود؛ در مقابل ممکن است رویدادها ابتدا به‌صورت خام نگهداری شوند و هر تیم بعداً خروجی مناسب خودش را بسازد.

تکمیل فنی: ETL و ELT دو الگوی معماری‌اند، نه دو دسته کاملاً جدا از محصول یا ابزار. یک خط لوله می‌تواند چند مرحله تبدیل قبل و بعد از Load داشته باشد. نگهداری داده خام نیز نیازمند سیاست امنیت، هزینه و مدت نگهداری است.

۳.۴ Change Data Capture یا CDC

CDC (ثبت تغییرات داده) روشی برای شناسایی و انتقال تغییرات منبع است. در نمایش کلاس، در رویکرد بازخوانی کامل، حتی وقتی فقط دو ردیف تغییر کرده‌اند، همه ردیف‌های جدول دوباره خوانده می‌شوند؛ اما در CDC هدف انتقال همان Insert, Update و Deleteهای مرتبط است. مدرس نمونه‌ای از CDC مبتنی بر لاگ پایگاه داده را شرح داد.

تکمیل فنی: تمام پیاده‌سازی‌های CDC الزاماً از Transaction Log استفاده نمی‌کنند؛ روش‌های مبتنی بر Trigger یا مقایسه دوره‌ای نیز وجود دارند. CDC نیز لزوماً «کاملاً آنی» نیست و تأخیر، ترتیب تغییرات، حذف رکورد و بازیابی پس از توقف باید بررسی شود.

۳.۵ Idempotency و Backfill

Idempotency یعنی اجرای تکراری یک عملیات، پس از بار نخست، اثر نهایی اضافه‌ای ایجاد نکند. مدرس دو سناریو نشان داد: در اولی اجرای دوباره باعث ایجاد سطر تکراری می‌شد؛ در دومی حتی با اجرای چندباره، نتیجه ذخیره‌شده یکسان می‌ماند. این ویژگی برای Retry و بازیابی خطا ضروری است.

> **مثال اضافی:** اگر هر بار فایل سفارش‌های روز قبل با روش Append بی‌قید بارگذاری شود، اجرای دوباره باعث Duplicate می‌شود. اگر به‌جای آن بر اساس شناسه سفارش `MERGE` یا Upsert صحیح اجرا شود، می‌توان نتیجه را تکرارپذیر کرد؛ البته رفتار حذف و تغییر سفارش هم باید تعریف شده باشد.

Backfill (پرکردن داده‌های جاافتاده) اجرای دوباره مسیر داده برای یک بازه گذشته است. در مثال مدرس، داده سه روز از یک API دریافت نشده بود، اما داده قبل و بعد از آن وجود داشت. خط لوله باید امکان دریافت دوباره فقط همان سه روز را داشته باشد.

- **تکمیل فنی:** Backfill امن باید با Idempotency همراه شود تا داده تاریخی دوباره شمرده نشود. دسترسی به داده گذشته در منبع، بازه زمانی دقیق و اثر Backfill بر گزارش‌های منتشرشده نیز باید بررسی شود.

۳.۶ Orchestration

Orchestration هماهنگ‌کردن اجرای کارها با توجه به ترتیب، زمان‌بندی، وابستگی و وضعیت موفقیت یا شکست آن‌هاست. مدرس Apache Airflow را به‌عنوان نمونه ارکستریتور خط لوله داده و Kubernetes را به‌عنوان نمونه‌ای از ارکستریشن در سطح زیرساخت معرفی کرد.

- **تکمیل فنی:** Airflow معمولاً زمان‌بندی و وابستگی Taskها را مدیریت می‌کند؛ Kubernetes عمدتاً اجرای Workloadهای کانتینری، مقیاس‌پذیری و وضعیت مطلوب زیرساخت را کنترل می‌کند. این دو می‌توانند در یک معماری هم‌زمان استفاده شوند.

۴. پردازش داده و معماری‌های جریانی

۴.۱ Batch Processing و Stream Processing

در Batch Processing داده‌های یک بازه جمع می‌شوند و به‌صورت دسته‌ای پردازش می‌شوند؛ مانند اجرای یک Job هر شب ساعت ۲ برای سفارش‌های روز گذشته. در Stream Processing رویدادها هنگام ورود یا با تأخیر کوتاه پردازش می‌شوند؛ مانند تحلیل پیوسته ثبت سفارش یا تغییر رفتار کاربران.

ویژگی	Batch	Stream
واحد ورودی	دسته‌ای از داده	جریان رویدادها
زمان اجرا	دوره‌ای یا بر اساس نیاز	پیوسته یا نزدیک به پیوسته
مزیت اصلی	سادگی نسبی و پردازش تجمیعی	تأخیر پایین در نتیجه
نکته مهم	تازگی داده	کنترل ترتیب، تکرار و داده دیررس

تکمیل فنی: Stream Processing لزوماً بدون تأخیر یا «دقیقاً در همان لحظه» نیست. نیاز واقعی را با هدف تأخیر مثل چند ثانیه یا چند دقیقه تعریف کنید. داده دیررس و Windowing می‌توانند نتیجه محاسبه را بعداً تغییر دهند.

۴.۲ معماری Kappa و Lambda

در توضیح مدرس، معماری Kappa زمانی مطرح شد که جریان رویداد به ستون فقرات پردازش تبدیل می‌شود؛ نمونه آن سامانه‌ای است که همه اطلاعات مورد نیاز را از رویدادهای پیوسته می‌گیرد. معماری Lambda دو مسیر Batch و Speed/Streaming را در کنار هم نگه می‌دارد تا داده‌ای که باید سریع پاسخ بدهد و داده‌ای که دیرتر و کامل‌تر می‌رسد هر دو پوشش داده شوند.

نمونه عملی مدرس از کارگزاری بود: معاملات کاربران در طول روز باید سریع پردازش می‌شدند، اما فایل نهایی سازمان مربوط در پایان روز دریافت می‌شد و برای اصلاح و تکمیل اطلاعات مبنا قرار می‌گرفت.

تکمیل فنی: Kappa به معنای ممنوع‌بودن هر نوع Batch نیست؛ ایده اصلی آن ساده‌سازی معماری با جریان قابل بازپردازش است. انتخاب میان Lambda, Kappa یا یک معماری ترکیبی به منبع داده، بازپردازش، هزینه نگهداری دو مسیر و نیاز تطبیق نتایج بستگی دارد.

۴.۳ Message Broker, Producer و Consumer

Message Broker واسطه‌ای برای دریافت، نگهداری و رساندن پیام‌ها به مصرف‌کنندگان است. Producer رویداد تولید می‌کند و Consumer به رویدادهای مورد نیاز گوش می‌دهد. در مثال واقعی کارگزاری، تیم معاملات و تیم پرتفوی داده‌های مربوط به خود را منتشر می‌کردند و تیم داده از آن‌ها برای پردازش استفاده می‌کرد.

مدرس Kafka و RabbitMQ را نام برد و به الگوی Publish/Subscribe نیز اشاره کرد. در این الگو Publisher رویداد را منتشر می‌کند و Subscriber موضوع مورد نیازش را دنبال می‌کند.

> **تکمیل فنی:** Kafka و RabbitMQ قابلیت‌ها و مدل تحویل یکسانی ندارند و Producer/Consumer همیشه دقیقاً معادل Publisher/Subscriber نیست. در Kafka ترتیب Offset در محدوده Partition معنا دارد، نه لزوماً در کل Topic.

۴.۴ Sink و Delivery Semantics

Sink یا Destination مقصدی است که نتیجه پردازش به آن ارسال می‌شود. این مقصد می‌تواند پایگاه داده، فایل، Topic دیگر یا سامانه تحلیلی باشد.

Delivery Semantics مشخص می‌کند هنگام خطا و Retry چه تضمینی برای رسیدن یا تکرارنشدن پیام داریم.

مدل تحویل	تضمین اصلی	ریسک
At-most-once	پیام حداکثر یک بار پردازش/تحویل می‌شود	احتمال ازدست‌رفتن پیام
At-least-once	Retry برای تحویل پیام وجود دارد	احتمال پردازش تکراری
Exactly-once	اثر پردازش، تحت شرایط تعریف‌شده، دقیقاً یک بار اعمال می‌شود	پیاده‌سازی و هماهنگی دشوارتر

- **تکمیل فنی:** Exactly-once تضمین سراسری و خودکار برای همه سامانه‌های متصل نیست. در یک خط لوله واقعی، قابلیت تراکنشی Sink، وضعیت پردازش و سازوکار Commit هم تعیین می‌کنند نتیجه نهایی دوباره ثبت شود یا نه.

۴.۵ Offset و Checkpoint

Offset نشانه موقعیت پیام در یک جریان مرتب است و کمک می‌کند بدانیم Consumer تا کجا پیش رفته است. اگر مصرف‌کننده از نقطه نامناسبی از سر بگیرد، ممکن است بعضی پیام‌ها تکرار یا جا بیفتند. مدرس مثال زد که اگر رویداد شماره ۵ هنوز پردازش نشده باشد و خواندن از ۶ شروع شود، داده ۵ از قلم می‌افتد.

Checkpoint تصویر کامل‌تری از وضعیت پردازش را برای بازیابی نگه می‌دارد؛ برای مثال ممکن است علاوه بر موقعیت ورودی، وضعیت یک شمارنده یا مجموع تجمعی را ثبت کند. دانستن Offset بدون داشتن State قبلی برای ادامه برخی محاسبات کافی نیست.

> **تکمیل فنی:** ثبت Offset قبل از تثبیت نتیجه در Sink می‌تواند باعث فقدان داده شود و ثبت آن بعد از نتیجه نیز در صورت Retry ممکن است خروجی تکراری بسازد. مرز Commit و Checkpoint باید با مدل تحویل خط لوله هماهنگ باشد.

۵. پایگاه داده و لایه‌های ذخیره‌سازی

۵.۱ SQL و پایگاه داده رابطه‌ای

SQL زبان رایج پرس‌وجو و مدیریت داده در پایگاه‌های رابطه‌ای است. پایگاه داده رابطه‌ای اطلاعات را در جدول‌هایی با ستون و ردیف نگهداری می‌کند و برای ارتباط بین جدول‌ها از کلیدها استفاده می‌شود. مدرس Oracle را نمونه‌ای از این خانواده معرفی کرد.

۵.۲ NoSQL و ACID در برابر BASE

NoSQL خانواده‌ای متنوع از پایگاه‌های داده با مدل‌های Document, Key-Value, Column-family و Graph است. مدرس فایل رویداد JSON را مثال زد که بعضی پیام‌ها ممکن است فیلدهای متفاوتی داشته باشند؛ نگهداری چنین داده‌ای در برخی مدل‌های NoSQL انعطاف‌پذیرتر است.

> - **تکمیل فنی:** SQL و NoSQL دو گروه کاملاً متضاد نیستند؛ برخی پایگاه‌های رابطه‌ای JSON را نگهداری و جست‌وجو می‌کنند و بعضی سامانه‌های NoSQL نیز تراکنش ارائه می‌دهند. «انعطاف Schema» به معنای بی‌نیازی از قرارداد داده و Validation نیست.

ACID مجموعه‌ای از ویژگی‌های تراکنش است: Atomicity (همه یا هیچ)، Consistency (حفظ قیود تعریف‌شده)، Isolation (جداسازی تراکنش‌های هم‌زمان) و Durability (ماندگاری تغییر تأییدشده). مدرس برای Atomicity مثال انتقال پول میان دو حساب را مطرح کرد: نباید پول از یک حساب کم شود اما همان تراکنش در حساب دیگر ثبت نشود.

BASE معمولاً به Basically Available, Soft State و Eventual Consistency اشاره می‌کند. برای توضیح تفاوت نیاز کسب‌وکار، مدرس مثال موجودی حساب بانکی و تعداد لایک یک پست را مقایسه کرد: در حساب بانکی ناهماهنگی می‌تواند آسیب جدی بزند، اما نمایش موقت تعداد لایک کمی قدیمی‌تر ممکن است قابل تحمل باشد.

- **تکمیل فنی:** Consistency در ACID یعنی تراکنش قیود پایگاه داده را نقض نکند؛ همان مفهوم Consistency در قضیه CAP نیست. همین‌طور انتخاب یک پایگاه داده NoSQL الزاماً به معنای کنارگذاشتن ACID یا پذیرش BASE نیست.

۵.۳ Data Warehouse و Data Lake

Data Warehouse (انبار داده) فضایی سازمان‌دهی‌شده برای تحلیل و گزارش‌گیری است. داده‌های منابع مختلف در آن با تعریف‌های مشترک تجمیع می‌شوند تا تیم‌ها برای یک شاخص کسب‌وکاری به نتیجه قابل تطبیق برسند. مدرس از مفهوم Single Source of Truth نام برد.

Data Lake (دریاچه داده) فضای ذخیره‌سازی انواع داده خام یا پردازش‌شده با قالب‌های گوناگون است. در تجربه مدرس، منابع شامل XML، فایل Excel، پایگاه داده و API بودند. اگر داده بدون شناسنامه، کنترل کیفیت و برنامه استفاده جمع شود، Data Lake ممکن است به Data Swamp تبدیل شود.

- **تکمیل فنی:** Single Source of Truth فقط با جمع‌آوری داده در یک مخزن ایجاد نمی‌شود؛ تعریف دقیق Metric، مالکیت داده، کنترل دسترسی و Reconciliation میان سیستم‌ها لازم است. یک Lake نیز بدون Catalog و سیاست Retention به‌سختی قابل استفاده خواهد بود.

۵.۴ Data Lakehouse

Lakehouse تلاش می‌کند انعطاف ذخیره‌سازی فایل‌ها را با امکانات مدیریت جدول و تحلیل ترکیب کند. مدرس Delta Lake و Apache Iceberg را مثال زد که روی لایه فایل و Storage, Metadata و قابلیت‌های جدول‌محور فراهم می‌کنند. برای اجرای Query ممکن است از یک Query Engine مستقل استفاده شود.

- **تکمیل فنی:** Delta Lake و Iceberg «خودِ موتور Query» نیستند؛ معمولاً قالب جدول و متادیتای لازم برای خواندن و نوشتن قابل اطمینان را فراهم می‌کنند. میزان کارایی به Engine, Layout فایل، نوع Workload و عملیات نگهداری بستگی دارد.

۵.۵ Row-oriented و Column-oriented Storage

در ذخیره‌سازی سطری، مقادیر یک رکورد کنار هم ذخیره می‌شوند؛ در ذخیره‌سازی ستونی، مقادیر یک ستون کنار هم قرار می‌گیرند. مدرس برای درخواست اطلاعات کامل یک مشتری، رویکرد سطری و برای گزارش تحلیلی روی شهر همه

مشتریان، رویکرد ستونی را توضیح داد. PostgreSQL را نمونه عمومی سامانه تراکنشی و ClickHouse را نمونه موتور تحلیلی ستونی معرفی کرد.

حالت	الگوی دسترسی مناسب	نمونه
Row-oriented	خواندن یا به‌روزرسانی رکورد مشخص	یافتن سفارش یک مشتری
Column-oriented	اسکن و تجمیع تعداد محدودی ستون	جمع فروش به تفکیک شهر

- **تکمیل فنی:** ستونی‌بودن همیشه سریع‌تر و سطری‌بودن همیشه کندتر نیست؛ ایندکس، اندازه داده، فشرده‌سازی، نوع Query و طراحی سیستم تعیین‌کننده‌اند. نتایج Benchmark را فقط با Dataset و تنظیمات یکسان مقایسه کنید.

۵.۶ قالب‌های CSV, JSON, Avro, Parquet و ORC

مدرس تفاوت فایل‌های متنی قابل خواندن با فرمت‌های دودویی مناسب پردازش را مطرح کرد. CSV و JSON معمولاً برای انسان قابل خواندن‌اند؛ Avro از قالب دودویی همراه Schema بهره می‌برد؛ Parquet و ORC نیز برای بسیاری از تحلیل‌های ستونی طراحی شده‌اند.

فرمت	مدل کلی	کاربرد متداول
CSV	متن جدولی	تبادل ساده داده تخت
JSON	متن ساختاری و تو در تو	API و رویداد
Avro	دودویی و مبتنی بر Schema	انتقال رکورد و پیام
Parquet	دودویی ستونی	تحلیل روی Data Lake
ORC	دودویی ستونی	ذخیره‌سازی تحلیلی

تکمیل فنی: «باینری» بودن به‌تنهایی دلیل سریع‌تر بودن نیست. برای تحلیل از Parquet/ORC، انتخاب ستون و فشرده‌سازی بسیار مهم است؛ برای پردازش تک‌رویداد Avro ممکن است مناسب‌تر باشد. اندازه فایل، Schema Evolution و خواننده مقصد را در نظر بگیرید.

۵.۷ Query Engine و کلاینت پایگاه داده

Query Engine دستور پرس‌وجو را برنامه‌ریزی و اجرا می‌کند. برخی پایگاه‌ها Engine داخلی دارند؛ در معماری Lakehouse ممکن است موتور جداگانه‌ای روی فایل‌ها و جدول‌ها Query اجرا کند. مدرس همچنین به ابزارهایی مثل DataGrip برای نوشتن و ارسال Query اشاره کرد.

> **تکمیل فنی:** DataGrip یک SQL Client/IDE است، نه Query Engine. Query در موتور پایگاه داده یا سامانه پردازش اجرا می‌شود و DataGrip رابط ارسال دستور و مشاهده نتیجه را فراهم می‌کند.

۶. طراحی مدل تحلیلی

۶.۱ Normalization و Denormalization

Normalization داده را در جدول‌هایی با رابطه مشخص سازمان‌دهی می‌کند تا تکرارهای غیرضروری و مشکلات درج، ویرایش و حذف کاهش یابند. مدرس مثال مشتری و سفارش را زد: اطلاعات مشتری یک‌بار در جدول مشتری ذخیره می‌شود و سفارش‌ها با کلید به او اشاره می‌کنند.

Denormalization می‌تواند عمداً برخی ویژگی‌ها را در خروجی تحلیلی تکرار کند تا تعداد Join یا هزینه محاسبه کاهش یابد. در مثال کلاس، شهر مشتری کنار هر سفارش قابل نمایش بود، اما در صورت تغییر شهر، باید درباره تاریخچه و به‌روزرسانی تصمیم گرفت.

> **تکمیل فنی:** تکرار داده در یک مدل تحلیلی الزاماً خطا نیست، اما ممکن است ناسازگاری ایجاد کند. در داده فروش تاریخی، «شهر مشتری در زمان سفارش» با «شهر فعلی مشتری» دو Metric/Attribute متفاوت هستند و نباید بدون تعریف با هم جایگزین شوند.

۶.۲ Data Modeling, Fact و Dimension

Data Modeling یعنی مشخص‌کردن موجودیت‌ها، روابط، کلیدها و Grain داده. در مدل تحلیلی، Fact معمولاً رخداد قابل تحلیل مثل فروش را در سطح مشخص نگه می‌دارد و Dimension زمینه آن را توضیح می‌دهد؛ برای مثال محصول، مشتری و تاریخ.

جزء	پاسخ به چه سوالی؟	نمونه
Fact Sales	چه رخداد یا عددی ثبت شد؟	تعداد و مبلغ هر قلم سفارش
Dim Product	کدام محصول؟	نام و دسته محصول
Dim Customer	چه مشتری‌ای؟	شناسه و ویژگی مشتری
Dim Date	چه زمانی؟	تاریخ، ماه و فصل

تکمیل فنی: هر Fact فقط شامل مقدار عددی نیست؛ رویدادها و Factهای بدون Measure نیز وجود دارند. پیش از تعریف کلید و Join باید Grain را صریح نوشت، مثلاً «هر ردیف برابر یک قلم سفارش». در غیر این صورت خطر Fan-out و دوباره‌شماری فروش وجود دارد.

۶.۳ Star Schema و Snowflake Schema

در Star Schema, Fact مرکزی مستقیماً به Dimensionهای مرتبط وصل می‌شود. در Snowflake Schema، برخی Dimensionها خودشان به جدول‌های توضیحی تفکیک می‌شوند؛ مانند Dimension محصول که به Dimension دسته‌بندی متصل شده است. مدرس توضیح داد که ساختار ستاره‌ای معمولاً ساده‌تر است و ساختار دانه‌برفی می‌تواند افزونگی را کاهش دهد.

تکمیل فنی: سرعت بیشتر Star و فضای کمتر Snowflake حکم قطعی نیست. کارایی به Query Optimizer، حجم داده، شیوه ذخیره و نیاز تحلیل بستگی دارد. در بسیاری از مدل‌های BI، سادگی روابط و وضوح Grain از صرفاً کم‌کردن تعداد جدول‌ها مهم‌تر است.

۶.۴ Slowly Changing Dimension یا SCD

SCD برای مدیریت تغییر ویژگی Dimension در گذر زمان است. مثال مدرس جابه‌جایی مشتری از تهران به شیراز بود: اگر شهر قدیمی را بازنویسی کنیم، ممکن است گزارش فروش تاریخی به‌اشتباه به شهر جدید نسبت داده شود.

در SCD Type 1 مقدار قبلی بازنویسی می‌شود و تاریخچه نگهداری نمی‌گردد. در SCD Type 2 ردیف نسخه جدیدی با بازه اعتبار ایجاد می‌شود تا بتوانیم وضعیت هر زمان را بازیابی کنیم.

مثال اضافی: در Type 2 می‌توان برای مشتری دو رکورد داشت: «تهران؛ از شروع تا ۰۱/۰۱/۱۴۰۴» و «شیراز؛ از ۰۱/۰۱/۱۴۰۴ به بعد». شرط Join فروش به Dimension باید علاوه بر شناسه مشتری، بازه اعتبار یا کلید نسخه درست را رعایت کند.

تکمیل فنی: در SCD Type 2 فقط اضافه‌کردن ستون‌های `valid_from` و `valid_to` کافی نیست؛ طراحی Surrogate Key، جلوگیری از بازه‌های هم‌پوشان و سازگاری Fact با نسخه درست Dimension نیز مهم‌اند.

۶.۵ Data Mart

Data Mart زیرمجموعه‌ای موضوعی از داده‌های تحلیلی است که برای نیاز یک واحد سازمانی تهیه می‌شود؛ مثلاً داده‌های ویژه تیم فروش یا مارکتینگ. مدرس اشاره کرد که این کار می‌تواند ارائه داده و کنترل دسترسی را برای هر تیم ساده‌تر کند.

۷. مقیاس‌پذیری و سامانه‌های توزیع‌شده

۷.۱ Distributed Systems و Cluster

در سیستم توزیع‌شده، چند ماشین یا Node در قالب یک سرویس یا کار واحد همکاری می‌کنند. Cluster مجموعه‌ای از منابع پردازشی یا ذخیره‌سازی است که با هماهنگی به هدف مشترک خدمت می‌کند. مدرس توضیح داد که بسیاری از مسائل Big Data با تقسیم کار و داده میان چند ماشین حل می‌شوند.

۷.۲ Vertical و Horizontal Scaling

Vertical Scaling یعنی قوی‌ترکردن یک ماشین با CPU, RAM یا منابع بیشتر. Horizontal Scaling یعنی اضافه‌کردن ماشین‌ها یا نمونه‌های سرویس. مدرس این دو الگو را در زمینه پایگاه داده‌های رابطه‌ای و سامانه‌های توزیع‌شده مقایسه کرد.

تکمیل فنی: پایگاه‌های رابطه‌ای نیز بسته به محصول و معماری می‌توانند Replication, Read Replica, Sharding یا توزیع بار داشته باشند. بنابراین جمله «SQL Server فقط عمودی مقیاس می‌گیرد» تعمیم درستی نیست؛ محدودیت دقیق به نوع عملیات خواندن/نوشتن و نسخه و معماری بستگی دارد.

۷.۳ Partitioning, Sharding و Replication

Partitioning تقسیم منطقی داده به بخش‌هایی کوچک‌تر است؛ مثلاً فایل‌های سفارش بر اساس تاریخ. با انتخاب کلید مناسب می‌توان در برخی Queryها فقط Partitionهای مرتبط را خواند و کار را سریع‌تر کرد.

Sharding به تقسیم داده میان چند Node اشاره می‌کند؛ برای مثال داده تاریخ‌های متفاوت روی ماشین‌های مختلف قرار بگیرند. Replication برعکس، نسخه‌هایی از همان داده را روی چند Node نگهداری می‌کند تا تحمل خطا یا ظرفیت خواندن بهتر شود.

مفهوم	ایده اصلی	مثال
Partitioning	تقسیم داده به بخش‌ها	پوشه‌بندی سفارش‌ها بر اساس روز
Sharding	توزیع بخش‌های متفاوت داده	تقسیم مشتریان میان سه Node
Replication	نگهداری نسخه‌های بیشتر از داده	دو Replica برای داده یکسان

تکمیل فنی: Partitioning همیشه معادل جداکردن داده روی ماشین‌های مختلف نیست؛ ممکن است همه Partitionها روی یک سیستم باشند. انتخاب کلید بد نیز باعث Skew و Partitionهای بسیار کوچک یا بزرگ می‌شود.

۷.۴ قضیه CAP و Eventual Consistency

CAP سه مفهوم Consistency, Availability و Partition Tolerance را در سیستم توزیع‌شده بررسی می‌کند. مدرس برای توضیح ناسازگاری موقت، مثال چند سرور را زد که یکی از آن‌ها هنوز مقدار به‌روز یک داده را دریافت نکرده است. Eventual Consistency یعنی اگر تغییر جدیدی رخ ندهد و ارتباط برقرار بماند، Replicaها در نهایت به وضعیت سازگار می‌رسند.

تکمیل فنی: صورت دقیق قضیه CAP درباره زمان وقوع Network Partition است: سامانه نمی‌تواند در همان شرایط هم تضمین سازگاری خطی همه عملیات و هم پاسخ‌گویی همیشگی را حفظ کند. عبارت رایج «از سه گزینه فقط دو تا انتخاب کن» ساده‌سازی گمراه‌کننده‌ای است؛ Partition Tolerance در سیستم توزیع‌شده واقعی معمولاً قابل حذف نیست.

تکمیل فنی: Consistency در CAP به رفتار مشاهده مقدار میان Nodeها اشاره می‌کند و با C موجود در ACID یکسان نیست. Eventual Consistency به معنی الزاماً «داده غلط» نیست؛ معنای دقیق، امکان مشاهده نسخه قدیمی‌تر تا زمان همگام‌شدن است.

۸.کیفیت، حاکمیت و عملیات داده

۸.۱ Data Quality

Data Quality کیفیت داده برای کاربرد تعریف‌شده را می‌سنجد. مدرس به ویژگی‌هایی مثل درستی، کامل‌بودن، یکتایی و به‌موقع‌بودن اشاره کرد؛ برای نمونه هر سفارش باید فقط یک‌بار در مجموعه معتبر ظاهر شود و گزارش روزانه در زمان وعده‌داده‌شده آماده باشد.

بُعد	سوال کنترلی	نمونه کنترل
Accuracy	مقدار با واقعیت سازگار است؟	تطبیق مبلغ سفارش با سیستم اصلی
Completeness	چیزی کم نشده؟	شمارش سفارش‌های روز
Uniqueness	داده ناخواسته تکرار نشده؟	یکتایی شناسه سفارش در Grain تعریف‌شده
Timeliness	داده به‌موقع آماده است؟	تأخیر آخرین Refresh
Consistency	تعریف و مقدار میان منابع سازگار است؟	یکسان‌بودن واحد پول و تعریف فروش
Validity	مقدار با قرارداد سازگار است؟	فرمت قابل قبول تاریخ

تکمیل فنی: وجود Duplicate همیشه به معنای خطا نیست؛ اگر Grain داده «هر قلم سفارش» باشد، تکرار `order_id` طبیعی است. کنترل یکتایی باید روی کلید مرتبط با Grain اعمال شود.

۸.۲ Reconciliation و مسیر عیب‌یابی داده

در پاسخ به سوال یکی از دانشجویان درباره اختلاف عدد انبار داده و سیستم اصلی، مدرس پیشنهاد کرد داده را در لایه‌های مختلف بررسی کنیم: ابتدا منبع اصلی، سپس لایه خامی که بدون تبدیل اساسی ثبت شده و در نهایت خروجی تبدیل‌شده انبار داده. به این ترتیب معلوم می‌شود اختلاف هنگام دریافت ایجاد شده یا در Transformation و Join.

مثال اضافی: برای بررسی اختلاف فروش یک روز، تعداد سفارش‌ها، تعداد شناسه‌های یکتا، جمع مبلغ و تعداد رکوردهای حذف‌شده را در Source, Raw و Warehouse با فیلتر زمانی یکسان محاسبه کنید. اگر Raw و Source برابرند ولی Warehouse فرق دارد، اول سراغ Transformation، فیلتر وضعیت و Fan-out بروید.

- **تکمیل فنی:** فقط Row Count کافی نیست؛ ممکن است تعداد ردیف یکسان ولی مقدارها اشتباه باشد. کنترل زمان استخراج، Time Zone، حذف سفارش، بازگشت وجه و تفاوت تعریف Metric هم ضروری است.

۸.۳ Governance, Lineage و Data Catalog

Data Governance چارچوب مالکیت، دسترسی، سیاست و مسئولیت داده است. Data Lineage مسیر داده را از منبع تا مقصد نشان می‌دهد: داده از کجا آمده، چه تبدیل‌هایی روی آن انجام شده و کدام گزارش‌ها به آن وابسته‌اند.

Data Catalog مثل فهرست کتابخانه به کاربران کمک می‌کند مجموعه‌داده قابل اعتماد را پیدا کنند؛ برای مثال بدانند جدول `sales_orders` چه معنایی دارد، مالک آن کیست و هر چند وقت به‌روزرسانی می‌شود. مدرس در این بخش بر اهمیت مالکیت کسب‌وکاری داده تأکید کرد.

۸.۴ DataOps و Version Control

در پایان جلسه به پیوند مهندسی داده با شیوه‌های مهندسی نرم‌افزار اشاره شد. DataOps به اتوماسیون، مشاهده‌پذیری، آزمون و مدیریت قابل اعتماد تغییرات در خط لوله داده مربوط است. Version Control نیز تغییر کد و فایل‌های پروژه را ثبت می‌کند تا همکاری، بازبینی و بازگشت به نسخه پیشین ممکن باشد؛ Git از ابزارهای رایج آن است.

- **تکمیل فنی:** نسخه‌بندی کد خط لوله با نسخه‌بندی داده یکی نیست. برای بازتولید یک گزارش ممکن است علاوه بر Commit کد، نسخه Schema, Snapshot داده، تنظیمات اجرا و وابستگی‌ها نیز لازم باشد.

۹. انتقال تجربه مدرس و تمرین

مدرس چند بار از تجربه کار در دیجی‌کالا و کارگزاری فارابی مثال زد: دریافت رویدادها برای پردازش جریان، لزوم استفاده هم‌زمان از خروجی روزانه و رویدادهای حین روز، اشتراک‌گذاری داده میان تیم‌ها از طریق Message

Broker، نگهداری فایل‌های ورودی متفاوت و بررسی اختلاف داده انبار داده با منابع اصلی. هدف مثال‌ها روشن‌کردن مسئله واقعی هر اصطلاح بود، نه تجویز یک معماری برای همه شرکت‌ها.

تمرین پیشنهادی مدرس برای درک ذخیره‌سازی: با داده یکسان، یک Query تحلیلی را در یک سامانه سطری و یک موتور ستونی اجرا و زمان پاسخ را در شرایط قابل مقایسه بررسی کنید. برای مسیر یادگیری نیز مراجعه مجدد به Roadmap مهندسی داده پیشنهاد شد.

۱۰. نقشه ذهنی جلسه و جمع‌بندی

مسیر پیشنهادی برای کنار هم قراردادن اصطلاح‌ها چنین است: Source و Generation → Ingestion و CDC → Batch یا Stream → Transformation و ETL/ELT → Storage مانند Warehouse, Lake و Lakehouse → Modeling و Query Engine → مصرف تحلیل و AI. روی همه این مسیر، Orchestration, Idempotency, Backfill, Data Quality, Monitoring, Lineage, Security و Governance نقش محافظتی و مدیریتی دارند.

نکته مرکزی جلسه این بود که اصطلاح‌ها را به صورت جدا از نیاز کسب‌وکار حفظ نکنیم. وقتی مسئله واقعی مشخص باشد، پرسش «داده با چه سرعتی می‌رسد؟»، «آیا باید دوباره‌پردازش شود؟»، «کدام تعریف تجاری را دنبال می‌کنیم؟» و «خطای سیستم چه خسارتی ایجاد می‌کند؟» راهنمای انتخاب ابزار و معماری خواهد بود.

۱۱. واژه‌نامه جلسه

اصطلاح انگلیسی	معادل یا توضیح فارسی
Data Engineering	مهندسی داده
Big Data	کلان‌داده
Metadata	فراداده
Data Pipeline	خط لوله داده
Ingestion	دریافت داده از منبع
Transformation	تبدیل و آماده‌سازی داده
ETL / ELT	الگوهای استخراج، تبدیل و بارگذاری
CDC	دریافت تغییرات داده
Idempotency	تکرارپذیری بدون اثر اضافی

اصطلاح انگلیسی	معادل یا توضیح فارسی
Backfill	بازپردازش داده تاریخی جاافتاده
Orchestration	هماهنگی و مدیریت اجرای کارها
Batch Processing	پردازش دسته‌ای
Stream Processing	پردازش جریانی
Kappa / Lambda	الگوهای معماری پردازش داده
Message Broker	واسطه پیام
Producer / Consumer	تولیدکننده و مصرف‌کننده پیام
Delivery Semantics	تضمین‌های تحویل یا پردازش پیام
Offset	موقعیت پیام در جریان مرتب
Checkpoint	نقطه ثبت وضعیت پردازش
Sink	مقصد دریافت خروجی
SQL / NoSQL	خانواده‌های مدل و دسترسی پایگاه داده
ACID / BASE	مفاهیم تراکنش و سازگاری توزیع‌شده
Data Warehouse	انبار داده
Data Lake	دریاچه داده
Lakehouse	معماری ترکیبی دریاچه و انبار داده
Parquet / ORC / Avro	قالب‌های ذخیره و انتقال داده
Query Engine	موتور اجرای پرس‌وجو
Normalization	نرمال‌سازی
Fact / Dimension	جدول رخداد و جدول توصیفی
Star / Snowflake Schema	الگوهای ستاره‌ای و دانه‌برفی
SCD	مدیریت تغییر تاریخی ابعاد
Data Mart	لایه داده ویژه یک حوزه کسب‌وکار
Scaling	افزایش ظرفیت سامانه
Partitioning / Sharding	بخش‌بندی و توزیع داده
Replication	نگهداری نسخه‌های تکراری داده
CAP	قضیه سازگاری، دسترس‌پذیری و تحمل جدایش
Data Quality	کیفیت داده

اصطلاح انگلیسی	معادل یا توضیح فارسی
Reconciliation	تطبیق داده میان مراحل
Data Lineage	تبارشناسی و مسیر داده
Data Catalog	فهرست اطلاعات مجموعه‌داده‌ها
Data Governance	حاکمیت داده
DataOps	عملیات مهندسی داده
Version Control	مدیریت نسخه تغییرات

۱۲. سوال‌های مرور

---

۱. مهم‌ترین مرحله‌های چرخه مهندسی داده چیست؟

۲. Metadata با محتوای خود داده چه تفاوتی دارد؟

۳. چرا Volume تنها معیار انتخاب ابزارهای Big Data نیست؟

۴. ETL و ELT چه تفاوتی در ترتیب کار دارند؟

۵. کاربرد CDC در مقایسه با بازخوانی کامل داده چیست؟

۶. چگونه Idempotency اجرای دوباره خط لوله را ایمن‌تر می‌کند؟

۷. Backfill در چه وضعیتی لازم می‌شود؟

۸. Offset و Checkpoint چه تفاوتی دارند؟

۹. در At-least-once چه مشکلی باید کنترل شود؟

۱۰. Data Warehouse, Data Lake و Lakehouse چه تفاوتی دارند؟

۱۱. چرا یک Query تحلیلی ممکن است روی ذخیره‌سازی ستونی مناسب‌تر باشد؟

۱۲. Grain چه نقشی در طراحی Fact و جلوگیری از Double Counting دارد؟

۱۳. در SCD Type 1 و Type 2 تغییر شهر مشتری چگونه ثبت می‌شود؟

۱۴. تفاوت Sharding با Replication چیست؟

۱۵. اگر فروش Warehouse با Source متفاوت باشد، برای یافتن منشأ اختلاف چه بررسی‌هایی انجام می‌دهید؟

۱۳. پاسخ‌نامه مختصر

۱. تولید، دریافت، تبدیل، نگهداری و ارائه برای مصرف‌کننده؛ با نقش‌های مشترک میان تیم‌ها.

۲. Metadata مشخصات، ساختار و زمینه داده را توصیف می‌کند، نه الزاماً مقدار اصلی آن را.

۳. سرعت، تنوع، SLA، هزینه و قابلیت ابزار نیز مؤثرند.

۴. ETL قبل از Load تبدیل می‌کند؛ ELT پس از بارگذاری اولیه.

۵. تغییرها را منتقل می‌کند و نیاز به انتقال دوباره همه داده را در بسیاری از سناریوها کاهش می‌دهد.

۶. Retry نباید رکورد یا اثر اضافی ایجاد کند؛ مثلاً با Upsert و کلید پایدار.

۷. وقتی بازه‌ای از داده گذشته ناقص، خراب یا دریافت‌نشده باشد.

۸. Offset جای پیام را نشان می‌دهد؛ Checkpoint می‌تواند State محاسبه را نیز نگه دارد.

۹. احتمال Duplicate؛ با طراحی Sink و عملیات Idempotent کنترل می‌شود.

۱۰. Warehouse برای مدل تحلیلی مشترک؛ Lake برای ذخیره انواع داده؛ Lakehouse برای ترکیب فایل و مدیریت جدول.

۱۱. تنها ستون‌های لازم را می‌خواند و از فشرده‌سازی و پردازش ستونی سود می‌برد.

۱۲. تعریف می‌کند هر ردیف نماینده چیست و مانع Join چندبرابرکننده می‌شود.

۱۳. Type 1 بازنویسی؛ Type 2 نگهداری نسخه‌های تاریخی با بازه اعتبار.

۱۴. Sharding داده متفاوت را تقسیم می‌کند؛ Replication کپی داده را نگه می‌دارد.

۱۵. تعریف Metric و زمان را یکسان کنید؛ Source, Raw و Warehouse را از نظر شمارش، مبلغ، یکتایی، فیلتر و Join تطبیق دهید.